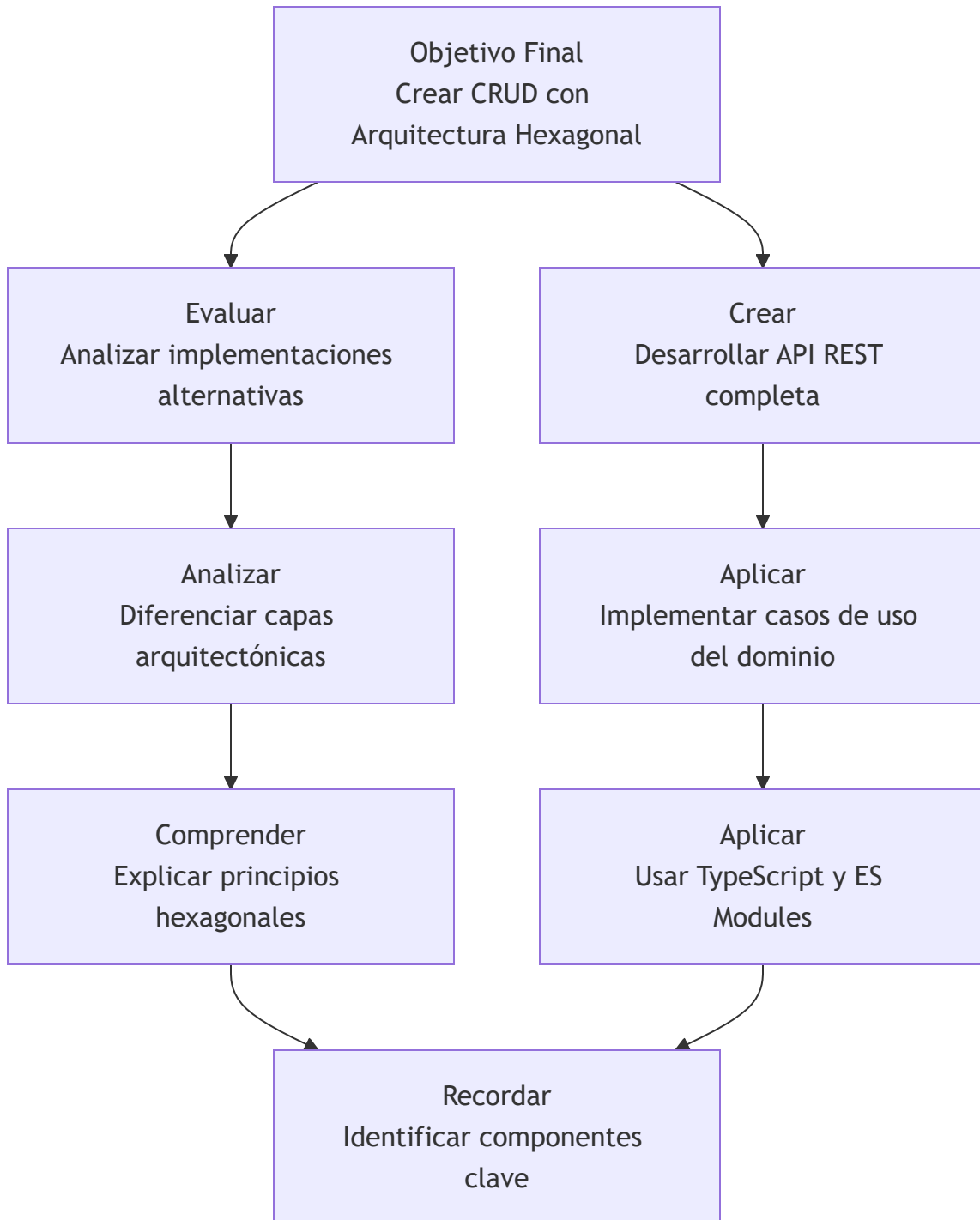


Guía 1: Narrativa del Proyecto - CRUD de Empleados con

Arquitectura Hexagonal (Windows)

1. Objetivos de Aprendizaje

Pirámide de Objetivos - Taxonomía de Bloom



Objetivos Específicos por Verbo

Nivel Cognitivo	Verbo	Objetivo
Recordar	Identificar	Reconocer los componentes principales de la arquitectura hexagonal
Recordar	Enumerar	Listar las capas de la arquitectura y sus responsabilidades
Comprender	Explicar	Describir el flujo de datos entre capas en la arquitectura
Comprender	Diferenciar	Distinguir entre entidades, casos de uso y adaptadores
Aplicar	Configurar	Establecer proyecto Node.js con TypeScript y ES Modules
Aplicar	Implementar	Codificar la estructura básica de cada capa arquitectónica
Analizar	Descomponer	Separar el problema en componentes hexagonales
Evaluar	Validar	Verificar que cada capa mantenga su responsabilidad única
Crear	Desarrollar	Construir API REST funcional siguiendo la arquitectura

2. Narrativa del Proyecto

El Contexto: Sistema de Gestión de Empleados

Imagina que trabajas en una empresa de desarrollo de software y te asignan crear un sistema interno para gestionar empleados. La empresa necesita:

1. **Registrar** nuevos empleados con sus datos básicos
2. **Consultar** la información de los empleados existentes
3. **Actualizar** datos cuando haya cambios
4. **Eliminar** empleados que ya no trabajan en la empresa
5. **Realizar operaciones específicas** como aumentar salarios

El desafío técnico: El cliente (en este caso, el departamento de RRHH) podría cambiar de opinión sobre:

- Cómo almacenar los datos (memoria → base de datos)

- Cómo exponer la API (REST → GraphQL)
- Qué reglas de negocio aplicar

La solución: Arquitectura Hexagonal que nos permite cambiar la infraestructura sin tocar el núcleo del negocio.

Historia de Usuario Principal

COMO administrador de RRHH

QUIERO gestionar los datos de los empleados

PARA mantener actualizada la información del personal

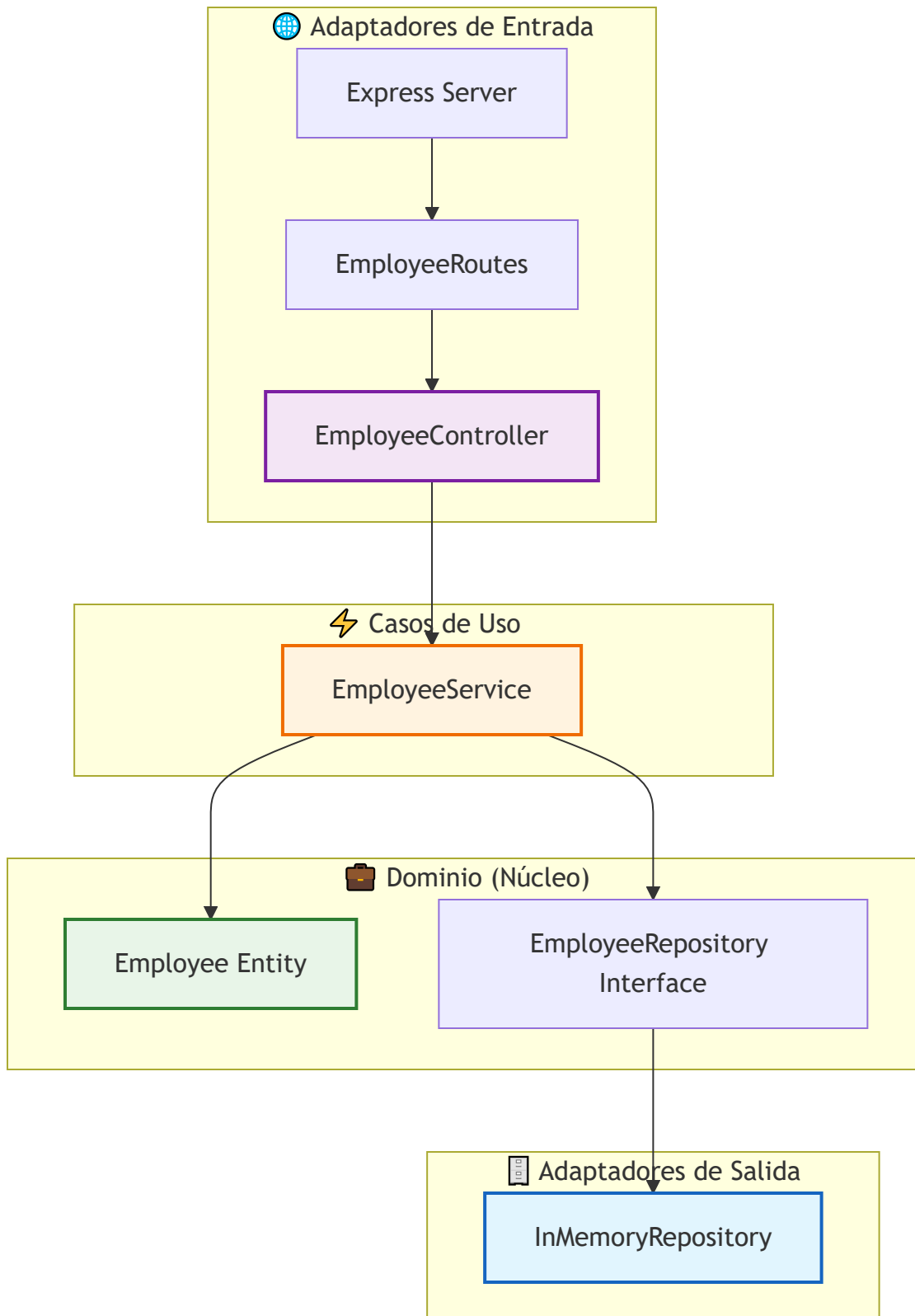
Criterios de Aceptación:

- Puedo crear un nuevo empleado con cédula, nombres y salario
- Puedo ver la lista completa de empleados
- Puedo buscar un empleado específico por ID
- Puedo actualizar los datos de un empleado
- Puedo eliminar empleados del sistema
- Puedo aumentar el salario de un empleado en porcentaje



3. Estructura del Proyecto

Diagrama de Arquitectura Hexagonal



Estructura de Directorios

employee-crud-hexagonal/

```
├──  src/ # Código fuente principal
│   ├──  domain/ #  NÚCLEO - Lógica de negocio
│   │   ├── employee.entity.ts # Entidad Empleado
│   │   └── employee.repository.ts # Interfaz del repositorio
│   │
│   ├──  application/ #  CASOS DE USO
│   │   └── employee.service.ts # Servicio con lógica de aplicación
│   │
│   ├──  infrastructure/ #  INFRAESTRUCTURA
│   │   ├── repositories/ # Implementaciones de repositorio
│   │   ├── controllers/ # Controladores HTTP
│   │   ├── routes/ # Rutas Express
│   │   └── server.ts # Configuración del servidor
│   │
│   └── app.ts #  Punto de entrada principal
├──  package.json #  Dependencias y scripts
├──  tsconfig.json #  Configuración TypeScript
├──  setup-directories.ps1 #  Script para crear estructura
├──  verify-structure.ps1 #  Script de verificación
└──  README.md #  Documentación
```

4. Instalación Paso a Paso (Windows)

4.1 Paso 1: Preparar PowerShell (Windows)

```
# Abrir PowerShell como Administrador:
# 1. Verificar versión de PowerShell
$PSVersionTable.PSVersion

# 2. Cambiar política de ejecución (solo una vez, si es necesario)
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser

# 3. Verificar Node.js instalado
node --version
# Debe mostrar versión 18 o superior

# 4. Verificar npm instalado
npm --version
```

4.2 Paso 2: Crear Directorio del Proyecto

```
# Navegar a donde quieres crear el proyecto (ej: Escritorio)
cd ~/Desktop

# Crear directorio del proyecto
mkdir employee-crud-hexagonal
cd employee-crud-hexagonal

# Verificar que estás en el directorio correcto
pwd
```

4.3 Paso 3: Inicializar Proyecto NPM

```
# Inicializar proyecto npm con valores por defecto
npm init -y

# Verificar que se creó package.json
Get-Content package.json
```

4.4 Paso 4: Configurar ES Modules en package.json

Editar package.json manualmente o ejecutar:

```
# Abrir package.json en editor (puedes usar Notepad)
notepad package.json

# O modificar con PowerShell:
$packageJson = Get-Content package.json -Raw | ConvertFrom-Json
$packageJson.type = "module"
$packageJson | ConvertTo-Json | Set-Content package.json
```

package.json debería quedar así:

```
{
  "name": "employee-crud-hexagonal",
  "version": "1.0.0",
  "description": "",
  "type": "module",
  "main": "dist/app.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```


4.5 Paso 5: Instalar Dependencias PASO A PASO

DEPENDENCIAS DE PRODUCCIÓN

1. Express - Framework web

`npm install express`

2. CORS - Middleware para Cross-Origin

`npm install cors`

DEPENDENCIAS DE DESARROLLO

3. TypeScript - Superset de JavaScript

`npm install --save-dev typescript`

4. Tipos para Express

`npm install --save-dev @types/express`

5. Tipos para Node.js

`npm install --save-dev @types/node`

6. Tipos para CORS

`npm install --save-dev @types/cors`

7. Nodemon - Recarga automática

`npm install --save-dev nodemon`

8. TS-Node - Ejecutar TypeScript directamente

`npm install --save-dev ts-node`

Verificar instalación

`npm list --depth=0`

4.6 Paso 6: Crear Script para Estructura de Directorios

```
# 📄 setup-directories.ps1 - Solo crea la estructura
# Crear directorios manualmente
mkdir src, src/domain, src/application, src/infrastructure, src/infrastructure/repositories, src/infrastructure/controllers, src/infrastructure/routes

# Verificar que se crearon
Get-ChildItem -Recurse

# Crear archivos manualmente
echo "// Entidad Employee - Capa de Dominio" > src/domain/employee.entity.ts
echo "// Interfaz del Repositorio - Puerto" > src/domain/employee.repository.ts
echo "// EmployeeService - Casos de Uso" > src/application/employee.service.ts
echo "// Repositorio en Memoria" > src/infrastructure/repositories/in-memory.repository.ts
echo "// Controlador HTTP" > src/infrastructure/controllers/employee.controller.ts
echo "// Rutas Express" > src/infrastructure/routes/employee.routes.ts
echo "// Configuración del Servidor" > src/infrastructure/server.ts
echo "// Punto de Entrada Principal" > src/app.ts

# Verificar archivos creados
Get-ChildItem -Recurse -File
```

Ejecutar el script:

```
# Dar permiso de ejecución (si es necesario)
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass

# Ejecutar script
.\setup-directories.ps1
```

4.7 Paso 7: Crear Script de Verificación

4.8 Paso 8: Crear tsconfig.json

```
```powershell
Crear tsconfig.json manualmente o usar este comando: {#crear-tsconfigjson-manualmente-o-usar-@
@'
{
 "compilerOptions": {
 "target": "ES2022",
 "module": "NodeNext",
 "moduleResolution": "NodeNext",
 "lib": ["ES2022"],
 "outDir": "./dist",
 "rootDir": "./src",
 "strict": true,
 "esModuleInterop": true,
 "skipLibCheck": true,
 "forceConsistentCasingInFileNames": true,
 "declaration": true,
 "declarationMap": true,
 "sourceMap": true
 },
 "include": ["src/**/*"],
 "exclude": ["node_modules", "dist"]
}
'@ | Set-Content tsconfig.json

Write-Host "✅ tsconfig.json creado" -ForegroundColor Green
```

## 4.9 Paso 9: Actualizar Scripts en package.json

```
@{
 "dev" = "nodemon src/app.ts"
 "build" = "tsc"
 "start" = "node dist/app.js"
 "setup" = "powershell -File .\\setup-directories.ps1"
 "verify" = "powershell -File .\\verify-structure.ps1"
}
```

## 4.10 Paso 10: Probar la Configuración

```
1. Probar TypeScript {#1-probar-typescript }
npx tsc --version
```

```
2. Probar compilación {#2-probar-compilación }
npx tsc --noEmit
```

```
4. Iniciar servidor en desarrollo (debería fallar porque falta código, pero probar) {#4-iniciar }
npm run dev
```

# 5. Archivos de Configuración Explicados

## 5.1 package.json - Explicación Detallada

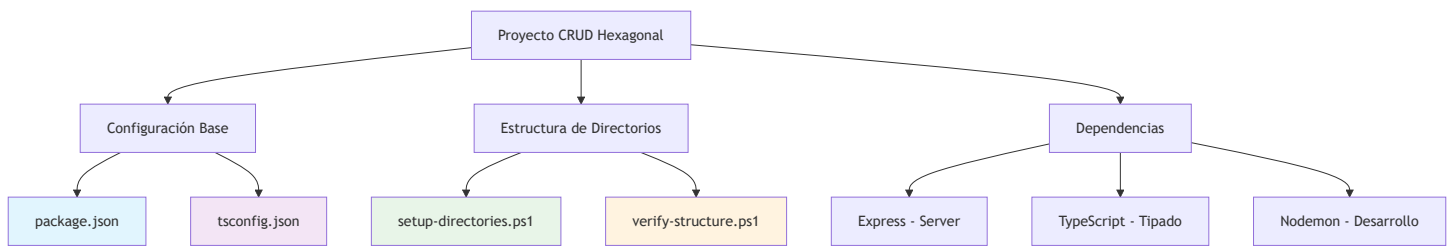
```
{
 "name": "employee-crud-hexagonal",
 "version": "1.0.0",
 "description": "CRUD de empleados con arquitectura hexagonal",
 "type": "module", // ← IMPORTANTE: Usa ES Modules
 "main": "dist/app.js",
 "scripts": {
 "dev": "nodemon src/app.ts", // Desarrollo con recarga automática
 "build": "tsc", // Compilar TypeScript a JavaScript
 "start": "node dist/app.js", // Ejecutar en producción
 "setup": "powershell -File .\\setup-directories.ps1", // Crear estructura
 "verify": "powershell -File .\\verify-structure.ps1" // Verificar estructura
 },
 "dependencies": {
 "express": "^4.18.2", // Framework web para crear API
 "cors": "^2.8.5" // Permitir peticiones desde otros dominios
 },
 "devDependencies": {
 "typescript": "^5.3.0", // Compilador TypeScript
 "@types/express": "^4.17.20", // Tipos para Express
 "@types/node": "^20.10.0", // Tipos para Node.js
 "@types/cors": "^2.8.16", // Tipos para CORS
 "nodemon": "^3.0.1", // Recarga automática al guardar
 "ts-node": "^10.9.1" // Ejecutar TypeScript sin compilar
 }
}
```

## 5.2 tsconfig.json - Configuraciones Clave

Configuración	Valor	¿Qué significa?	¿Por qué es importante?
target	ES2022	Versión de JavaScript	Usa características modernas
module	NodeNext	Sistema de módulos	Permite import/export (ES Modules)
moduleResolution	NodeNext	Cómo buscar módulos	Encuentra archivos .ts y .js

Configuración	Valor	¿Qué significa?	¿Por qué es importante?
outDir	./dist	Donde va el código compilado	Separa código fuente de compilado
rootDir	./src	Donde está el código fuente	Organización clara del proyecto
strict	true	Modo estricto de TypeScript	Más seguridad, menos errores
esModuleInterop	true	Compatibilidad entre módulos	Mejora interoperabilidad

### 5.3 Estructura Visual de Configuración



## 6. Verificación de la Configuración

## 4. Scripts disponibles

```
Write-Host "`n4. SCRIPTS NPM DISPONIBLES:" -ForegroundColor Yellow
npm run
```

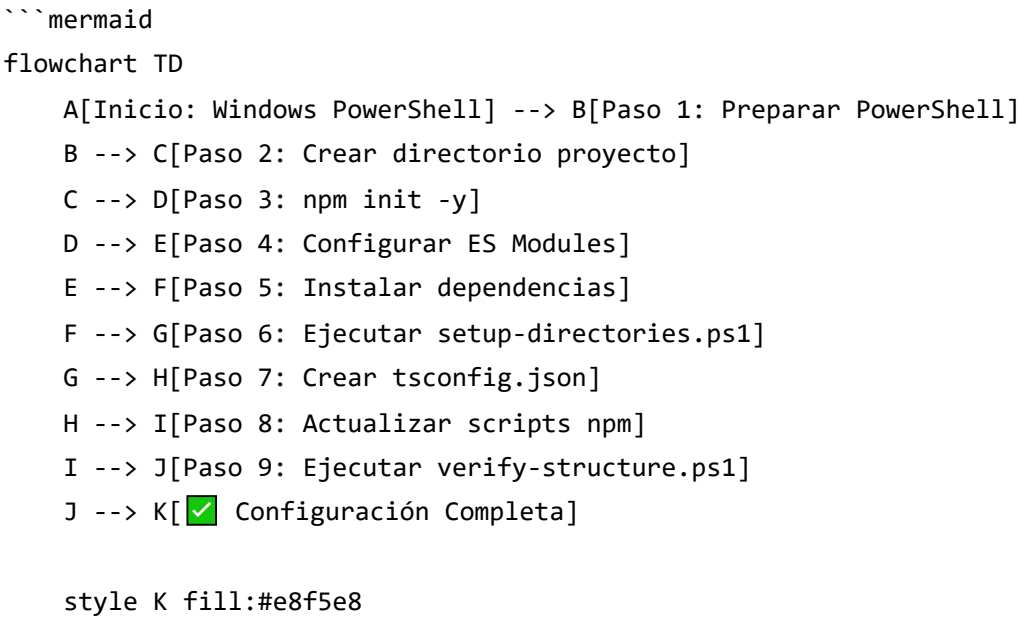
### \*\*Problemas Comunes y Soluciones (Windows)\*\* {#problemas-comunes-y-soluciones-windows }

Problema	Síntoma	Solución Windows
-----	-----	-----
**Error de ejecución de PowerShell**	"No se puede cargar el archivo"	`Set-ExecutionPolicy
**Nodemon no recarga**	Cambios no se reflejan	Usar `npm run dev` (no `node` directamente)
**Error de módulo ES**	"Cannot use import statement"	Verificar que `package.json` tenga `
**TypeScript no encuentra archivos**	"Cannot find module"	Verificar rutas en `tsconfig.js
**CORS bloqueado**	No puede acceder desde navegador	Asegurar middleware `cors()` en server

---

##  \*\*7. Flujo de Trabajo para Estudiantes\*\* {#-7-flujo-de-trabajo-para-estudiantes }

### \*\*Diagrama de Proceso de Configuración\*\* {#diagrama-de-proceso-de-configuración }



## 8. Revisiónr

### Puntos de Evaluación de la Configuración

Criterio	Puntos	Cómo verificar
Estructura de directorios	20	npm run verify o revisar manualmente

Criterio	Puntos	Cómo verificar
<b>Configuración TypeScript</b>	20	Verificar tsconfig.json y que <code>npx tsc --noEmit</code> funcione
<b>Dependencias instaladas</b>	20	<code>npm list --depth=0</code> muestra todas las dependencias
<b>ES Modules configurado</b>	20	package.json tiene <code>"type": "module"</code>
<b>Scripts npm funcionando</b>	20	<code>npm run dev</code> inicia servidor (aunque falle por código faltante)

## Archivos Esperados al Final de esta Guía

employee-crud-hexagonal/

```

├─ 📄 package.json # ✅ Con type: module y scripts
├─ 📄 tsconfig.json # ✅ Configuración TypeScript correcta
├─ 📄 setup-directories.ps1 # ✅ Script para crear estructura
├─ 📄 verify-structure.ps1 # ✅ Script de verificación
├─ 📄 README.md # ✅ (Opcional) Documentación
├─ 📁 node_modules/ # ✅ Dependencias instaladas
└─ 📁 src/ # ✅ Estructura hexagonal
 ├─ 📁 domain/
 │ ├─ employee.entity.ts # ✅ (vacío)
 │ └─ employee.repository.ts # ✅ (vacío)
 ├─ 📁 application/
 │ └─ employee.service.ts # ✅ (vacío)
 ├─ 📁 infrastructure/
 │ ├─ 📁 repositories/
 │ │ └─ in-memory.repository.ts # ✅ (vacío)
 │ └─ 📁 controllers/
 │ └─ employee.controller.ts # ✅ (vacío)
 │ └─ 📁 routes/
 │ └─ employee.routes.ts # ✅ (vacío)
 │ └─ server.ts # ✅ (vacío)
 └─ app.ts # ✅ (vacío)

```





## 9. Próximos Pasos

### Guía 2 - Capa de Dominio:

- Implementar entidad Employee con reglas de negocio
- Crear interfaz del repositorio (puerto)
- Validaciones básicas y métodos de dominio

### Tarea para estudiantes:

1. Ejecutar `npm run verify` y capturar pantalla del resultado
2. Investigar sobre ES Modules vs CommonJS
3. Revisar la documentación de TypeScript sobre interfaces y clases

## ? Preguntas de Reflexión para Estudiantes

1. ¿Por qué es importante la línea `"type": "module"` en `package.json`?
2. ¿Qué ventajas tiene separar el código en `src/` y el compilado en `dist/`?
3. ¿Cómo ayuda la arquitectura hexagonal cuando cambiamos de memoria a base de datos?
4. ¿Por qué instalamos `@types/express` además de `express`?

### ✅ Configuración Completada:

- ✅ Entorno Windows preparado
- ✅ Proyecto npm inicializado con ES Modules
- ✅ Dependencias instaladas paso a paso
- ✅ Estructura hexagonal creada con scripts
- ✅ TypeScript configurado correctamente
- ✅ Scripts de verificación implementados